let we make 1 practically and mathematically to understand better this deep learning,

Building AI with - Multiple Negatives Ranking (MNR)

The Training Data

- Pair 1: Anchor 1: "Python programming" | Positive 1: "Writing code in Python"
- Pair 2: Anchor 2: "Baking a cake" | Positive 2: "Making desserts in the oven"
- Pair 3: Anchor 3: "Playing basketball" | Positive 3: "Scoring points on the court"

Convert Sentences into Numbers

Our 12-Word Vocabulary List & Index Mapping:

1. python (Index 0)

2. programming (Index 1)

3. writing (Index 2)

4. code (Index 3)

5. baking (Index 4)

6. cake (Index 5)

7. making (Index 6)

8. desserts (Index 7)

9. oven (Index 8)

10. playing (Index 9)

11. basketball (Index 10)

12. scoring (Index 11)

• Anchor 1: "Python programming"

- [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

• Positive 1: "Writing code in Python"

- [1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]

• Anchor 2: "Baking a cake"

- [0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0]

• Positive 2: "Making desserts in the oven"

- [0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0]

• Anchor 3: "Playing basketball"

- [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0]

• Positive 3: "Scoring points on the court"

- [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

Initial Random Weights ($\boldsymbol{W}1$ and W2)
To turn your exact single-layer example into a **Deep Learning** model, we need to split your single 12×3 weight matrix into **two separate layers** connected by an activation function (ReLU), and then trace the forward pass and weight updates mathematically.

Step 1: Define the New Deep Architecture & Weights

Instead of a single 12×3 matrix, we will use two matrices:

1. **Hidden Layer Weight Matrix (W_1):** Size 12×2 (12 vocabulary words × 2 hidden nodes). We will use a small, clean set of random weights for our vocabulary words so we can do the math by hand.
2. **Output Layer Weight Matrix (W_2):** Size 2×3 (2 hidden nodes × 3 final dimensions X, Y, Z).

Let's define $\boldsymbol{W_1}$ $\boldsymbol{(12 \times 2)}$ for our vocabulary words:

Index	Word	Semantic Topic	W1 Weight Vector (2 Hidden Nodes)
0	python	Coding	[0.20, -0.10]
1	programming	Coding	[-0.15, 0.30]
2	writing	Coding	[0.35, -0.15]
3	code	Coding	[-0.10, 0.25]
4	baking	Baking	[0.10, 0.40]
5	cake	Baking	[-0.05, 0.35]

Index	Word	Semantic Topic	W1 Weight Vector (2 Hidden Nodes)
6	making	Baking	[0.15, 0.30]
7	desserts	Baking	[-0.20, 0.45]
8	oven	Baking	[0.05, 0.38]
9	playing	Sports	[0.45, -0.05]
10	basketball	Sports	[0.40, 0.05]
11	scoring	Sports	[0.38, -0.02]

Let's define W_2 (2×3):

- **Hidden Node 1:** $[0.50, -0.20, 0.30]$
- **Hidden Node 2:** $[-0.10, 0.40, 0.20]$

Step 2: Forward Pass for Anchor 1 ("Python programming")

1. Input Vector (X):

Anchor 1 contains *python* (index 0) and *programming* (index 1).

$$X = [1,1,0,0,0,0,0,0,0,0,0,0]$$

2. Multiply Input by W_1 (To get Hidden Layer Raw Scores):

We multiply our input vector by W_1 (which means we just add Row 0 and Row 1 of W_1 since their inputs are 1):

- **Hidden Node 1 Calculation:** $(1 \times 0.20) + (1 \times -0.15) = \mathbf{0.05}$
- **Hidden Node 2 Calculation:** $(1 \times -0.10) + (1 \times 0.30) = \mathbf{0.20}$

Raw Hidden Output = $[0.05, 0.20]$

3. Apply ReLU (Activation Function):

ReLU keeps positive numbers as they are and turns negatives into 0. Since both 0.05 and 0.20 are positive, they pass straight through:

$$\text{Hidden Activations } (H) = [0.05, 0.20]$$

4. Multiply Hidden Activations by W_2 (To get Final 3D Vector):

Now we take our hidden vector $[0.05, 0.20]$ and multiply it by W_2 (2×3):

- **X Coordinate:** $(0.05 \times 0.50) + (0.20 \times -0.10) = 0.025 - 0.020 = \mathbf{0.005}$
- **Y Coordinate:** $(0.05 \times -0.20) + (0.20 \times 0.40) = -0.010 + 0.080 = \mathbf{0.070}$
- **Z Coordinate:** $(0.05 \times 0.30) + (0.20 \times 0.20) = 0.015 + 0.040 = \mathbf{0.055}$

Final 3D Vector for Anchor 1: $[0.005, 0.070, 0.055]$

(You would repeat this exact same forward pass for Positive 1, Anchor 2, etc., to build your Similarity Grid, compute Softmax, and find your Batch Loss).

Generating 3D Vectors

 Anchor 1 ("Python programming"): [0.005, 0.070, 0.055]

 Positive 1 ("Writing code in Python"): [0.225, -0.090, 0.135]

 Anchor 2 ("Baking a cake"): [-0.050, 0.290, 0.165]

 Positive 2 ("Making desserts in the oven"): [-0.113, 0.452, 0.226]

 Anchor 3 ("Playing basketball"): [0.425, -0.170, 0.255]

 Positive 3 ("Scoring points on the court"): [0.190, -0.076, 0.114]

Similarity Grid
To build the **Similarity Grid** for your deep learning model, we calculate the dot product between every Anchor and every Positive in the batch (3×3 grid).

Here are your final 3D vectors again for reference:

- **Anchor 1 (A_1):** [0.005, 0.070, 0.055] | **Positive 1 (P_1):** [0.225, -0.090, 0.135]
- **Anchor 2 (A_2):** [-0.050, 0.290, 0.165] | **Positive 2 (P_2):** [-0.113, 0.452, 0.226]
- **Anchor 3 (A_3):** [0.425, -0.170, 0.255] | **Positive 3 (P_3):** [0.190, -0.076, 0.114]

Step-by-Step Dot Product Calculations

Row 1: Anchor 1 vs All Positives

- $A_1 \cdot P_1$**:** $(0.005 \times 0.225) + (0.070 \times -0.090) + (0.055 \times 0.135) = 0.001125 - 0.006300 + 0.007425 = \mathbf{0.0023}$
- $A_1 \cdot P_2$**:** $(0.005 \times -0.113) + (0.070 \times 0.452) + (0.055 \times 0.226) = -0.000565 + 0.031640 + 0.012430 = \mathbf{0.0435}$
- $A_1 \cdot P_3$**:** $(0.005 \times 0.190) + (0.070 \times -0.076) + (0.055 \times 0.114) = 0.000950 - 0.005320 + 0.006270 = \mathbf{0.0019}$

Row 2: Anchor 2 vs All Positives

- $A_2 \cdot P_1$**:** $(-0.050 \times 0.225) + (0.290 \times -0.090) + (0.165 \times 0.135) = -0.011250 - 0.026100 + 0.022275 = \mathbf{-0.0151}$
- $A_2 \cdot P_2$**:** $(-0.050 \times -0.113) + (0.290 \times 0.452) + (0.165 \times 0.226) = 0.005650 + 0.131080 + 0.037290 = \mathbf{0.1740}$

- $A_2 \cdot P_3$: $(-0.050 \times 0.190) + (0.290 \times -0.076) + (0.165 \times 0.114) = -0.009500 - 0.022040 + 0.018810 = \mathbf{-0.0127}$

Row 3: Anchor 3 vs All Positives

- $A_3 \cdot P_1$: $(0.425 \times 0.225) + (-0.170 \times -0.090) + (0.255 \times 0.135) = 0.095625 + 0.015300 + 0.034425 = \mathbf{0.1454}$
- $A_3 \cdot P_2$: $(0.425 \times -0.113) + (-0.170 \times 0.452) + (0.255 \times 0.226) = -0.048025 - 0.076840 + 0.057630 = \mathbf{-0.0672}$
- $A_3 \cdot P_3$: $(0.425 \times 0.190) + (-0.170 \times -0.076) + (0.255 \times 0.114) = 0.080750 + 0.012920 + 0.029070 = \mathbf{0.1227}$

The Final Similarity Grid (3×3)

$$\text{Similarity Grid} = \begin{bmatrix} \mathbf{0.0023} & 0.0435 & 0.0019 \\ -0.0151 & \mathbf{0.1740} & -0.0127 \\ 0.1454 & -0.0672 & \mathbf{0.1227} \end{bmatrix}$$

Loss Function and Penalty Calculation

Step 1: Divide the Similarity Grid by Temperature ($\boldsymbol{\tau} = \mathbf{0.8}$)

Here is **Step 1** of the Loss Function calculation: dividing every score in our Similarity Grid by the temperature ($\tau = 0.8$).

The Similarity Grid

$$\text{Similarity Grid} = \begin{bmatrix} 0.0023 & 0.0435 & 0.0019 \\ -0.0151 & 0.1740 & -0.0127 \\ 0.1454 & -0.0672 & 0.1227 \end{bmatrix}$$

Step-by-Step Division by $\boldsymbol{\tau = 0.8}$

Row 1 (Anchor 1 vs P1, P2, P3)

- $0.0023/0.8 = \mathbf{0.0029}$
- $0.0435/0.8 = \mathbf{0.0544}$
- $0.0019/0.8 = \mathbf{0.0024}$

Row 2 (Anchor 2 vs P1, P2, P3)

- $-0.0151/0.8 = \mathbf{-0.0189}$
- $0.1740/0.8 = \mathbf{0.2175}$
- $-0.0127/0.8 = \mathbf{-0.0159}$

Row 3 (Anchor 3 vs P1, P2, P3)

- $0.1454/0.8 = \mathbf{0.1818}$
- $-0.0672/0.8 = \mathbf{-0.0840}$
- $0.1227/0.8 = \mathbf{0.1534}$

The Resulting Scaled Scores Matrix

$$\text{Scaled Scores} = \begin{bmatrix} 0.0029 & 0.0544 & 0.0024 \\ -0.0189 & 0.2175 & -0.0159 \\ 0.1818 & -0.0840 & 0.1534 \end{bmatrix}$$

Step 2: Apply Softmax (Row-by-Row Probability Matrix)

Here is **Step 2: Apply Softmax** to turn our scaled scores into a row-by-row probability matrix where every row sums to 1.0 (or 100%).

The Softmax formula for each score x in a row is:

$$P(x) = \frac{e^x}{\sum(e^{\text{all scores in row}})}$$

Row-by-Row Softmax Calculations

Row 1: Scaled Scores [0.0029, 0.0544, 0.0024]

1. Exponentiate (e^{score}):
   - $e^{0.0029} \approx 1.0029$
   - $e^{0.0544} \approx 1.0559$
   - $e^{0.0024} \approx 1.0024$
   - Sum of exponentials = $1.0029 + 1.0559 + 1.0024 = 3.0612$
2. Normalize (divide each by the sum):
   - $P_{1,1} = 1.0029/3.0612 = \mathbf{0.33}$
   - $P_{1,2} = 1.0559/3.0612 = \mathbf{0.34}$
   - $P_{1,3} = 1.0024/3.0612 = \mathbf{0.33}$
   - *Row 1 Probabilities:* [0.33, 0.34, 0.33] (Sums to 1.0)

Row 2: Scaled Scores [-0.0189, 0.2175, -0.0159]

1. Exponentiate (e^{score}):
   - $e^{-0.0189} \approx 0.9813$
   - $e^{0.2175} \approx 1.2430$
   - $e^{-0.0159} \approx 0.9842$
   - Sum of exponentials = $0.9813 + 1.2430 + 0.9842 = 3.2085$
2. Normalize:
   - $P_{2,1} = 0.9813/3.2085 = \mathbf{0.31}$
   - $P_{2,2} = 1.2430/3.2085 = \mathbf{0.39}$
   - $P_{2,3} = 0.9842/3.2085 = \mathbf{0.30}$

- *Row 2 Probabilities:* [0.31, 0.39, 0.30] (Sums to 1.0)

Row 3: Scaled Scores [0.1818, -0.0840, 0.1534]

1. Exponentiate (e^{score}):
   - $e^{0.1818} \approx 1.1994$
   - $e^{-0.0840} \approx 0.9194$
   - $e^{0.1534} \approx 1.1658$
   - Sum of exponentials = $1.1994 + 0.9194 + 1.1658 = 3.2846$
2. Normalize:
   - $P_{3,1} = 1.1994/3.2846 = \mathbf{0.37}$
   - $P_{3,2} = 0.9194/3.2846 = \mathbf{0.28}$
   - $P_{3,3} = 1.1658/3.2846 = \mathbf{0.35}$
   - *Row 3 Probabilities:* [0.37, 0.28, 0.35] (Sums to 1.0)

The Final Probability Matrix

$$\text{Probability Matrix} = \begin{bmatrix} \mathbf{0.33} & 0.34 & 0.33 \\ 0.31 & \mathbf{0.39} & 0.30 \\ \mathbf{0.37} & 0.28 & 0.35 \end{bmatrix}$$

Step 4: Calculating the Loss (The Penalty)

Here is **Step 4: Calculating the Loss (The Penalty)** using the diagonal probabilities from our Softmax Probability Matrix.

In Multiple Negatives Ranking (MNR), we look at how confident the model was in choosing the **correct diagonal matches** (Anchor 1 with Positive 1, Anchor 2 with Positive 2, and Anchor 3 with Positive 3). We use **Negative Log-Likelihood ($-\ln$)** to measure the penalty.

The Diagonal Probabilities

From our Probability Matrix, our correct matching pairs are:

- **Anchor 1 ($P_{1,1}$):** 0.33
- **Anchor 2 ($P_{2,2}$):** 0.39
- **Anchor 3 ($P_{3,3}$):** 0.35

Step-by-Step Loss Calculations ($-\ln(\text{Probability})$)

1. **For Anchor 1:**
   - $-\ln(0.33) \approx \mathbf{1.11}$

(The model was only 33% *sure, so it receives a penalty of* 1.11*)*

2. **For Anchor 2:**
   - $-\ln(0.39) \approx \mathbf{0.94}$

(The model was 39% sure, resulting in a penalty of 0.94)

3. **For Anchor 3:**
    - $-\ln(0.35) \approx \mathbf{1.05}$

(The model was 35% sure, resulting in a penalty of 1.05)

Total Batch Loss (The Average Penalty)

To find the final batch loss, we add them up and divide by the number of pairs (3):

$$\text{Total Batch Loss} = \frac{1.11 + 0.94 + 1.05}{3} = \frac{3.10}{3} \approx \mathbf{1.03}$$

What does this mean?

Your **Total Batch Loss is** 1.03.

Because your deep learning model started with random weights in both W_1 and W_2, it is essentially guessing blindly (around 33% confidence per row), which creates a high error penalty.

The next step in the loop would be **Backpropagation**, where this loss value generates the gradients that push backward through W_2 and W_1, updating all the weights so that the model's confidence on the diagonal increases and the loss drops toward 0!

Compute Gradients

Step 1: The Softmax Gradient Formula (Cross-Entropy / MNR)

For a batch with Multiple Negatives Ranking (MNR) loss, the gradient of the loss with respect to our similarity scores is wonderfully simple: **It is just the Softmax Probability Matrix minus the Target Identity Matrix ($P - Y$).**

- P**:** Our Softmax Probability Matrix.
- Y**:** The target correct answers (a matrix with 1s on the diagonal for our true pairs, and 0s everywhere else).

$$\text{Target Identity Matrix } (Y) = \begin{bmatrix} \mathbf{1} & 0 & 0 \\ 0 & \mathbf{1} & 0 \\ 0 & 0 & \mathbf{1} \end{bmatrix}$$

Step 2: Subtracting Y from P ($P - Y$)

Let's calculate the gradient for each row by subtracting 1 from the correct diagonal probability:

Row 1 (Anchor 1)

- Probability: [0.33, 0.34, 0.33]
- Target: [1, 0, 0]
- Gradient Calculation:
    - $0.33 - 1 = \mathbf{-0.67}$
    - $0.34 - 0 = \mathbf{0.34}$

- $0.33 - 0 = \mathbf{0.33}$

- *Row 1 Gradient:* [-0.67, 0.34, 0.33]

Row 2 (Anchor 2)

- Probability: [0.31, 0.39, 0.30]
- Target: [0, 1, 0]
- Gradient Calculation:
  - $0.31 - 0 = \mathbf{0.31}$
  - $0.39 - 1 = \mathbf{-0.61}$
  - $0.30 - 0 = \mathbf{0.30}$
- *Row 2 Gradient:* [0.31, -0.61, 0.30]

Row 3 (Anchor 3)

- Probability: [0.37, 0.28, 0.35]
- Target: [0, 0, 1]
- Gradient Calculation:
  - $0.37 - 0 = \mathbf{0.37}$
  - $0.28 - 0 = \mathbf{0.28}$
  - $0.35 - 1 = \mathbf{-0.65}$
- *Row 3 Gradient:* [0.37, 0.28, -0.65]

The Final Score Gradient Matrix (3×3)

$$\text{Gradient Matrix} = \begin{bmatrix} \mathbf{-0.67} & 0.34 & 0.33 \\ 0.31 & \mathbf{-0.61} & 0.30 \\ 0.37 & 0.28 & \mathbf{-0.65} \end{bmatrix}$$

Adjusting the Input Weight using Gradient Descent for Python

To update both W_2 (the output layer matrix, 2×3) and W_1 (the input layer matrix, 12×2) using Gradient Descent (with learning rate $\eta = 0.1$), we follow the chain rule of **Backpropagation**.

Because the word **python** appears in both **Anchor 1** and **Positive 1**, its weight row in W_1 will receive gradient updates from both sentences!

Step 1: Updating W_2 (The Output Layer Matrix, Size 2×3)

To update W_2, we combine our hidden layer activations (H) with our Similarity Gradients ($P - Y$).

- **What W_2 does:** It connects our 2 hidden nodes to the 3 output dimensions (X, Y, Z).
- **The Gradient for W_2:** Summing across our batch, the calculated gradient matrix for W_2 is:

$$\text{Gradient for } W_2 = \begin{bmatrix} -0.1322 & 0.2368 & 0.0535 \\ 0.3658 & -0.5434 & -0.0673 \end{bmatrix}$$

- **Applying Gradient Descent ($W_{\text{new}} = W_{\text{old}} - (\eta \times \text{Gradient})$):**
    - *Old W_2 (Hidden Node 1):* [0.50, -0.20, 0.30]
    - *Calculation:* $0.50 - (0.1 \times -0.1322) = 0.50 + 0.0132 = \mathbf{0.5132}$
    - *Updated W_2 (Hidden Node 1):* [0.5132, -0.2237, 0.2947]

Step 2: Passing Error Backward to the Hidden Layer

Before we can update W_1, we must push the error backward through W_2 to find out how much each hidden node contributed to the mistake.

- We multiply the output gradients by W_2^T and factor in the derivative of the **ReLU** activation function. This tells us the exact error present at our 2 hidden nodes for Anchor 1 and Positive 1.

Step 3: Updating W_1 (Focusing on the word *python*, Index 0)

Now we compute the gradient for W_1 by multiplying our **Input Vector** by the hidden error vector.

Because *python* is present in **Anchor 1** and **Positive 1**, its weight row (Row 0) receives contributions from both sentences:

- **Calculated Gradient Vector for *python* (Index 0):**

$$\text{Grad}_{\text{python}} = [\mathbf{0.0117}, \mathbf{0.1179}]$$

Now, we apply the Gradient Descent update formula to the old weight for *python* ($W_{\text{old}} = [0.20, -0.10]$):

1. **Calculate the adjustment step ($\eta = 0.1$):**
    - $0.1 \times 0.0117 = \mathbf{0.0012}$
    - $0.1 \times 0.1179 = \mathbf{0.0118}$
2. **Subtract from the old weight:**
    - $X_{\text{new}} = 0.20 - 0.0012 = \mathbf{0.1988}$
    - $Y_{\text{new}} = -0.10 - 0.0118 = \mathbf{-0.1118}$

The Resulting Updated Weight for *python*

- **Old Weight (W_0):** [0.20, -0.10]
- **New Adjusted Weight (W_0):** [0.1988, -0.1118]

Final Result

Here are the complete, exact **Old** and **New Adjusted Weights** for both W_1 (input layer, 12×2) and W_2 (output layer, 2×3) after completing one full forward-and-backward gradient descent update step (with learning rate $\eta = 0.1$).

1. Output Layer Weight Matrix (W_2: Size 2×3)

Hidden Node	Old Weight (X,Y,Z)	New Adjusted Weight (X,Y,Z)
Hidden Node 1	[0.5000, -0.2000, 0.3000]	[0.5132, -0.2237, 0.2946]
Hidden Node 2	[-0.1000, 0.4000, 0.2000]	[-0.1366, 0.4543, 0.2067]

2. Input Layer Weight Matrix (W_1: Size 12×2)

Index	Word	Semantic Topic	Old Weight (2 Hidden Nodes)	New Adjusted Weight (2 Hidden Nodes)
0	**python**	Coding	[0.2000, -0.1000]	[0.1988, -0.1118]
1	**programming**	Coding	[-0.1500, 0.3000]	[-0.1381, 0.2882]
2	**writing**	Coding	[0.3500, -0.1500]	[0.3370, -0.1500]
3	**code**	Coding	[-0.1000, 0.2500]	[-0.1130, 0.2500]
4	**baking**	Baking	[0.1000, 0.4000]	[0.0819, 0.4204]
5	**cake**	Baking	[-0.0500, 0.3500]	[-0.0681, 0.3704]
6	**making**	Baking	[0.1500, 0.3000]	[0.1500, 0.3122]
7	**desserts**	Baking	[-0.2000, 0.4500]	[-0.2000, 0.4622]
8	**oven**	Baking	[0.0500, 0.3800]	[0.0500, 0.3922]
9	**playing**	Sports	[0.4500, -0.0500]	[0.4566, -0.0500]
10	**basketball**	Sports	[0.4000, 0.0500]	[0.4066, 0.0500]

Index	Word	Semantic Topic	Old Weight (2 Hidden Nodes)	New Adjusted Weight (2 Hidden Nodes)
11	**scoring**	Sports	[0.3800, -0.0200]	[0.4071, -0.0200]

What happens next?

You have successfully completed **one full epoch** of a Deep Learning training loop!

If you plug these new W_1 and W_2 matrices back into the beginning and run the forward pass again:

1. The new 3D vectors for your sentences will shift slightly closer to their correct matching pairs.
2. The diagonal probabilities in your Softmax matrix will rise.
3. The total batch loss will drop from 1.03 down a bit further.

By repeating this loop 50 to 100 times, both matrices will fully converge, clustering coding words, baking words, and sports words into their own distinct regions of your deep neural network!

Internal calculations

To get the exact 2×3 gradient matrix for W_2:

$$\text{Gradient for } W_2 = \begin{bmatrix} -0.1322 & 0.2368 & 0.0535 \\ 0.3658 & -0.5434 & -0.0673 \end{bmatrix}$$

We use the **Chain Rule of Backpropagation**. Because W_2 is used to calculate **both** Anchor vectors ($A = H_A W_2$) and Positive vectors ($P = H_P W_2$), the error flows into W_2 from two directions at once.

The full formula for the gradient of W_2 is:

$$\frac{\partial L}{\partial W_2} = (H_A^T \times dA) + (H_P^T \times dP)$$

Let's break down and calculate every piece of this equation step-by-step.

Step 1: Gather the Hidden Activations (H_A and H_P)

These are the outputs of Hidden Layer 1 after applying ReLU for all 3 pairs (shape 3×2):

- **Anchor Hidden Activations (H_A):**
  - A_1: [0.05, 0.20]

- A_2: [0.05, 0.75]
- A_3: [0.85, 0.00]

$$\text{Matrix } H_A = \begin{bmatrix} 0.05 & 0.20 \\ 0.05 & 0.75 \\ 0.85 & 0.00 \end{bmatrix}$$

- **Positive Hidden Activations (H_P):**
  - P_1: [0.45, 0.00]
  - P_2: [0.00, 1.13]
  - P_3: [0.38, 0.00]

$$\text{Matrix } H_P = \begin{bmatrix} 0.45 & 0.00 \\ 0.00 & 1.13 \\ 0.38 & 0.00 \end{bmatrix}$$

Step 2: Calculate Error Gradients for Vectors (dA and dP)

We take our Similarity Gradient Matrix (grad_sim from the previous step) and propagate it back to the Anchor and Positive vectors:

$$\text{grad_sim} = \frac{\text{Probability Matrix} - \text{Target Matrix}}{\text{Temperature } (\tau = 0.8)}$$

- $dA = \text{grad_sim} \times P$ (shape 3×3)
- $dP = \text{grad_sim}^T \times A$ (shape 3×3)

$$\text{Matrix } dA = \begin{bmatrix} -0.1601 & 0.2394 & 0.0306 \\ 0.2454 & -0.4097 & -0.0777 \\ -0.0900 & 0.1784 & 0.0488 \end{bmatrix}$$

$$\text{Matrix } dP = \begin{bmatrix} 0.1707 & -0.0256 & 0.1332 \\ 0.1892 & -0.2514 & -0.0134 \\ -0.3598 & 0.2769 & -0.1198 \end{bmatrix}$$

Step 3: Compute Term 1 ($H_A^T \times dA$)

This calculates how much the Anchor side contributed to the weight errors in W_2:

$$\text{Term 1} = H_A^T \times dA = \begin{bmatrix} 0.05 & 0.05 & 0.85 \\ 0.20 & 0.75 & 0.00 \end{bmatrix} \times \begin{bmatrix} -0.1601 & 0.2394 & 0.0306 \\ 0.2454 & -0.4097 & -0.0777 \\ -0.0900 & 0.1784 & 0.0488 \end{bmatrix}$$

$$\text{Term 1} = \begin{bmatrix} -0.0723 & 0.1431 & 0.0391 \\ 0.1520 & -0.2594 & -0.0522 \end{bmatrix}$$

Step 4: Compute Term 2 ($H_P^T \times dP$)

This calculates how much the Positive side contributed to the weight errors in W_2:

$$\text{Term 2} = H_P^T \times dP = \begin{bmatrix} 0.45 & 0.00 & 0.38 \\ 0.00 & 1.13 & 0.00 \end{bmatrix} \times \begin{bmatrix} 0.1707 & -0.0256 & 0.1332 \\ 0.1892 & -0.2514 & -0.0134 \\ -0.3598 & 0.2769 & -0.1198 \end{bmatrix}$$

$$\text{Term 2} = \begin{bmatrix} -0.0599 & 0.0937 & 0.0144 \\ 0.2137 & -0.2840 & -0.0152 \end{bmatrix}$$

Step 5: Add Term 1 and Term 2 Together

Combining both influences gives the final 2×3 gradient matrix for W_2:

$$\text{Gradient for } W_2 = \text{Term 1} + \text{Term 2}$$

$$\text{Gradient for } W_2 = \begin{bmatrix} -0.0723 + (-0.0599) & 0.1431 + 0.0937 & 0.0391 + 0.0144 \\ 0.1520 + 0.2137 & -0.2594 + (-0.2840) & -0.0522 + (-0.0152) \end{bmatrix}$$

$$\text{Gradient for } W_2 = \begin{bmatrix} \mathbf{-0.1322} & \mathbf{0.2368} & \mathbf{0.0535} \\ \mathbf{0.3658} & \mathbf{-0.5434} & \mathbf{-0.0673} \end{bmatrix}$$

Internal Calculation 2

To get the final gradient vector for the word **python**—which came out to $[\mathbf{0.0117}, \mathbf{0.1179}]$—we look at how error flows backward into W_1 from every sentence where the word *python* appears.

Because the word **python** is present in both **Anchor 1** and **Positive 1**, its weight row receives gradient contributions from **both** sentences. We calculate the hidden error vector for each sentence and then add them together.

Step 1: Find the Hidden Error Vector from Anchor 1 (A_1)

When we take the error from Anchor 1 (dA_1) and push it backward through W_2^T (and multiply by the ReLU derivative), we get the hidden layer error contribution for Anchor 1:

- **Anchor 1 Hidden Error Vector:** $[\mathbf{-0.1187}, \mathbf{0.1179}]$
- *Meaning:* This tells us how much the two hidden nodes erred during Anchor 1 ("Python programming").

Step 2: Find the Hidden Error Vector from Positive 1 (P_1)

Similarly, when we take the error from Positive 1 (dP_1) and push it backward through W_2^T (and apply the ReLU derivative), we get the hidden layer error contribution for Positive 1:

- **Positive 1 Hidden Error Vector:** $[\mathbf{0.1304}, \mathbf{0.0000}]$
- *Note on ReLU:* The second hidden node for Positive 1 had a raw value ≤ 0 after ReLU, so its derivative became 0, wiping out its error contribution (hence 0.0000).

Step 3: Add Them Together (Multi-Sentence Accumulation)

Because the input value for *python* is 1 in Anchor 1 and 1 in Positive 1, its total gradient is simply the sum of both hidden error vectors:

$$\text{Gradient}_{\text{python}} = (\text{Anchor 1 Error}) + (\text{Positive 1 Error})$$

$$\text{Gradient}_{\text{python}} = [-0.1187 \quad 0.1179] + [0.1304 \quad 0.0000]$$

$$\text{Gradient}_{\text{python}} = [-0.1187 + 0.1304 \quad 0.1179 + 0.0000] = [\mathbf{0.0117} \quad \mathbf{0.1179}]$$

Internal Calculation 3

To understand how we got the Anchor 1 hidden error vector **[-0.1187, 0.1179]**, we need to trace the error backward from the **final 3D output** back down into the **2 hidden nodes** using matrix transposition and the ReLU derivative.

Step 1: Gather the Ingredients for Anchor 1

1. **The Output Error for Anchor 1 (dA_1):**

This is the error vector at the final 3D coordinates (X, Y, Z) for Anchor 1 ("Python programming"), calculated from our similarity gradients:

$$dA_1 = [\mathbf{-0.1601} \quad \mathbf{0.2394} \quad \mathbf{0.0306}]$$

(Meaning: The model's final X, Y, Z coordinates for Anchor 1 were too low/high by these exact error amounts).

2. **The Output Weight Matrix Transposed (W_2^T):**

During the forward pass, W_2 (2×3) mapped our 2 hidden nodes *forward* to the 3D output.

To push errors *backward*, we use its transpose W_2^T (3×2):

$$W_2^T = \begin{bmatrix} 0.50 & -0.10 \\ -0.20 & 0.40 \\ 0.30 & 0.20 \end{bmatrix}$$

*(Column 1 represents the weights for **Hidden Node 1**, and Column 2 represents the weights for **Hidden Node 2**).*

Step 2: Push the Error Backward ($\boldsymbol{dA_1 \times W_2^T}$)

To find out how much the error at the output affected each hidden node, we take the dot product of our output error (dA_1) and W_2^T:

Calculating Error for Hidden Node 1:

We multiply dA_1 by Column 1 of W_2^T:

$$\text{Hidden Node 1 Error} = (-0.1601 \times 0.50) + (0.2394 \times -0.20) + (0.0306 \times 0.30)$$

$$\text{Hidden Node 1 Error} = -0.0800 - 0.0479 + 0.0092 = \mathbf{-0.1187}$$

Calculating Error for Hidden Node 2:

We multiply dA_1 by Column 2 of W_2^T:

$$\text{Hidden Node 2 Error} = (-0.1601 \times -0.10) + (0.2394 \times 0.40) + (0.0306 \times 0.20)$$

$$\text{Hidden Node 2 Error} = 0.0160 + 0.0958 + 0.0061 = \mathbf{0.1179}$$

So, before activation checks, our raw hidden error vector is:

$$\text{Raw Hidden Error} = [-0.1187 \quad 0.1179]$$

Step 3: Apply the ReLU Derivative

In deep learning, if a hidden node was turned "off" by ReLU during the forward pass (i.e., its raw value was ≤ 0), it cannot take any blame for the error, so its gradient must be wiped out to 0.

- During our forward pass for Anchor 1, the raw values for our 2 hidden nodes were [0.05, 0.20].
- Both values are greater than 0, meaning **both hidden nodes were active**.
- The derivative for active ReLU nodes is simply 1.

Multiplying our raw hidden error by the ReLU derivative ($[1,1]$) leaves the numbers unchanged:

$$\text{Final Hidden Error Vector for Anchor 1} = [-0.1187 \quad 0.1179]$$

What does this vector actually mean?

- **-0.1187** tells us how much **Hidden Node 1** contributed to the mistake in Anchor 1.
- **0.1179** tells us how much **Hidden Node 2** contributed to the mistake in Anchor 1.

Once we have this hidden error vector, we multiply it by the original input words (*python* and *programming*) to update W_1!